



Discrete optimization

The distributed flow shop scheduling problem with inter-factory transportation

Tristan Becker^{a,*}, Janis Neufeld^{b,c}, Udo Buscher^b^a TU Dresden, Faculty of Business and Economics, Junior Professorship of Management Science, Germany^b TU Dresden, Faculty of Business and Economics, Chair of Industrial Management, Germany^c Otto-von-Guericke University Magdeburg, Chair of Operations Management, Germany

ARTICLE INFO

Keywords:

Scheduling
Distributed flow shop
Inter-factory transportation
Combinatorial optimization
Heuristic algorithms

ABSTRACT

Large manufacturing companies often manage a network of multiple factories, creating a distributed flow shop scheduling problem for flowline manufacturing processes. This problem involves assigning jobs to one of several distributed factories, each equipped with identical flow shops, and completing the jobs within their designated factory. We expand upon the traditional distributed flow shop scheduling problem by incorporating the transportation of intermediate goods between factories. This transportation can occur after each machine, with shipping times dependent on the distance between the origin and destination factories, potentially delaying further processing. Our objective is to minimize the makespan across all factories. We introduce an iterated greedy search procedure specifically designed for distributed flow shop scheduling with inter-factory transportation. Based on a graph representation, a speed-up procedure is developed to improve the algorithms search ability. Through computational studies, we demonstrate the effectiveness and efficiency of our proposed algorithm, and provide insights into the benefits of transportation. Our findings indicate that incorporating transportation enhances machine utilization across factories by alleviating bottlenecks, ultimately improving overall efficiency and reducing makespan. Besides, we are able to quantify the trade-off between transport times and the benefits of inter-factory transportation.

1. Introduction

The Distributed Flow Shop Scheduling Problem (DFSP) is a well-studied and complex combinatorial optimization problem that arises in a variety of real-world applications, including but not limited to manufacturing, logistics, and service industries (Naderi & Ruiz, 2010). The problem deals with the scheduling of jobs in a distributed production environment, which comprises multiple identical flow shops. Each flow shop consists of a series of machines arranged in a specific order, with each machine responsible for a different operation on the jobs. In the DFSP, jobs have to be assigned to one of the distributed flow shops and cannot change between the flow shops during production. As such, the problem consists of determining both the optimal job-to-flow shop assignment and the job sequence within each flow shop. The most prevalent objective is to minimize the makespan across all flow shops (Lohmer & Lasch, 2021).

The distributed flow shop environment is a generalization of the well-known flow shop scheduling problem which is known to be NP-hard (Garey et al., 1976). Therefore, finding good schedules even for small-sized applications remains challenging, requiring specialized

heuristics. In this course, the DFSP has been an active field of research since its inception by Naderi and Ruiz (2010). Most studies on the DFSP include the permutation constraint, such that the order of jobs does not change between machines. Extensive research has been conducted on this problem, with various solution approaches proposed, including exact algorithms and heuristics (Fernandez-Viagas & Framinan, 2014; Naderi et al., 2023; Ruiz et al., 2019).

In many industrial settings, the production process is not isolated within individual flow shops. Instead, intermediate goods can be transported between different facilities. The transportation of intermediate goods may play a critical role in determining the efficiency and effectiveness of the production process. For example, in semiconductor manufacturing, jobs may be transported between different factories to balance workloads and improve resource utilization. This aspect has already been emphasized in the context of multi-site supply chain planning (Moon et al., 2002; Thoney et al., 2002), but has not been considered explicitly in distributed scheduling.

We propose a new variant of the DFSP that considers the transportation of intermediate goods between the distributed flow shops,

* Corresponding author.

E-mail address: msci@tu-dresden.de (T. Becker).

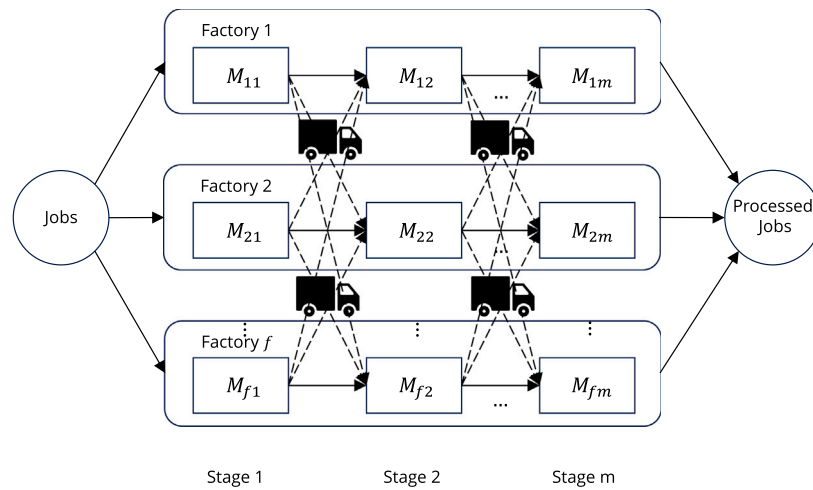


Fig. 1. Illustration of the distributed flow shop scheduling problem with inter-factory transportation.

allowing the transportation to occur between machines. This new variant accounts for the transportation of intermediate goods between the respective flow shops, possibly delaying further processing. The new variant of the DFSP allows for a more flexible production process, as it accounts for the possibility of transporting intermediate goods between machines of different flow shops. This flexibility can lead to better utilization of resources, reduced idle times, and more efficient production schedules. Furthermore, it provides a realistic representation of many modern real-world multi-site production systems, where transports between factories are possible.

The introduction of intermediate goods transportation adds a new layer of complexity to the DFSP, presenting new challenges. Fig. 1 illustrates our extension of the Distributed Flow Shop Scheduling Problem, where dashed lines represent transportation options introduced by inter-factory transportation. In addition to assigning jobs to flow shops and determining their processing sequence, the transportation moves of intermediate goods have to be considered. As pointed out by a recent review, there is no study on the transportation of intermediate goods in the context of distributed flow shop scheduling (Perez-Gonzalez & Framinan, 2023). Therefore, this study investigates the benefit of transporting intermediate goods as part of distributed flow shop scheduling. We propose a mathematical programming and constraint programming formulation for the Distributed Flow Shop Scheduling Problem With Inter-Factory Transportation (DFSPT). Furthermore, we propose an innovative heuristic for the DFSPT based on the ideas of iterated greedy search (Ruiz & Stützle, 2007). It is based on an efficient graph representation of the problem, which allows the development of a tailored speed-up procedure making use of critical paths. The computational analysis allows a quantification of the effects of inter-factory transportation on makespan reduction in different scenarios.

The next section presents our literature review. Section 1.2 outlines the contributions of this study, while we detail the organization of this paper in Section 1.3.

1.1. Literature overview

The focus of our literature review is on Distributed Flow Shop Scheduling Problems (DFSP) which is the subject of our extension. In particular, we provide an overview of the literature on the Distributed Permutation Flow Shop Scheduling Problem (DPFSP) since the existing literature typically employs the permutation constraint (pmu). Our review does not aim to be exhaustive such that we refer the interested reader to Lohmer and Lasch (2021) and Perez-Gonzalez and Framinan (2023) for recent reviews. This review primarily addresses exact and heuristic solution approaches for the DPFSP, as well as extensions, with a special emphasis on transportation and supply chain aspects.

1.1.1. Distributed permutation flow shop scheduling

The DPFSP was formally introduced by Naderi and Ruiz (2010) as an extension of the well-known Permutation Flow Shop Scheduling Problem (PFSP). It emerges from the reality of today's business operations where numerous factories within a supply chain have replaced the single factory model. Since its inception, the literature on the DPFSP has seen a rapid expansion (Perez-Gonzalez & Framinan, 2023).

Due to its relative simplicity, the makespan objective has been the predominant objective function under study in the existing research. Recently, total completion time has also begun to attract attention (Fernandez-Viagas et al., 2018). Other objectives, including total flowtime (see, e.g., Mao et al., 2022), total tardiness, the number of tardy jobs (see, e.g., Wang et al., 2017), and maximum lateness (see, e.g., Cai et al., 2017) have been investigated, albeit less extensively. Several studies have incorporated multiple objectives (see, e.g., Chen et al., 2019) or cost-related objective functions (see, e.g., Cai et al., 2017; Rifai et al., 2016). Due to the significant challenges of the makespan objective, we focus on makespan in this study.

Several extensions of the DPFSP have been introduced, including blocking and buffer-related constraints (see, e.g., Ribas et al., 2017), no-wait problems (see, e.g., Shao et al., 2017), no-idle problems (see, e.g., Li, Pan et al., 2021), and setup time constraints (see, e.g., Guo et al., 2022). Notably, the most attention has been dedicated to blocking and buffer-related constraints. In recent years, studies have also begun to consider uncertainties and heterogeneous DPFSP, although these areas remain relatively limited (Wang et al., 2017, 2016).

In terms of exact solution methods, Mixed Integer Linear Programming (MILP) and Constraint Programming (CP) formulations have been applied to the DPFSP where many studies on approximate methods also include a MILP formulation. Exact methods for the DPFSP are typically only able to solve smaller instances to optimality and do not scale well to larger problem instances, even when using state-of-the-art commercial solvers (Naderi & Ruiz, 2010). Because of this limitation, research on exact methods remains scarce (Hamzadayi, 2020). Recently, a study by Naderi et al. (2023) compared the performance of MILP and CP formulations for several scheduling problems, including the DPFSP. They found that CP formulations significantly outperformed MILP formulations for the DPFSP in terms of solution quality. However, the majority of the research on solution approaches has dealt with the development of approximate methods capable of handling larger, more realistically sized instances (Perez-Gonzalez & Framinan, 2023). We give an overview of the literature on approximate methods for the DPFSP with makespan objective in the next section.

1.1.2. Approximate methods

Naderi and Ruiz (2010) proposed a Variable Neighborhood Search along with their statement of the DPFSP and identified the NEH heuristic as a suitable initialization procedure. Subsequently, several studies improved on the initial contribution using Genetic Algorithms and Tabu Search (Gao & Chen, 2011a; Gao et al., 2013, 2012). Furthermore, Gao and Chen (2011b) proposed an enhancement of the NEH method of Naderi and Ruiz (2010). A substantial variety of metaheuristics has been applied to the DPFSP, including Artificial Bee Colony, Evolutionary and Genetic Algorithms, Iterated Greedy Algorithms, and Iterated Local Search Procedures, Variable and Large Neighborhood Search, and Tabu Search (see, e.g., Gao & Chen, 2011a; Gao et al., 2013; Naderi & Ruiz, 2010; Pan et al., 2019; Ruiz et al., 2019).

There has been a profound interest in the application of Iterated Greedy Algorithms (IG) to the DPFSP, with the first application by Lin et al. (2013). The extension prompted the proposal of various IG algorithms (see, e.g., Fernandez-Viagas & Framinan, 2014; Ruiz et al., 2019). IG algorithms have gained popularity due to their simplicity in implementation and adaptability to new problem types, while often providing competitive performance in comparison to other solution approaches.

In general, approximate methods typically utilize a composite dispatch rule or an extension of the NEH heuristic to obtain a starting solution, which is then enhanced via the application of a specific metaheuristic (Perez-Gonzalez & Framinan, 2023). The current state-of-the-art in DPFSP algorithms remains undefined owing to the existence of several algorithms which have not been compared with one another (Bargauoui et al., 2017; Ren et al., 2021; Ruiz et al., 2019).

Common heuristic solution procedures for DPFSP often exploit decompositions. After jobs have been allocated to specific factories, each factory's flow shop scheduling problem can be resolved individually. The solution usually comprises one permutation per factory, although alternate solution representations have been implemented. In such heuristics, efforts are primarily focused on the factory determining the makespan, either by redistributing jobs or intensifying the search on the factory permutation. However, these approaches cannot be employed for transportation. If jobs are shifted between factories, the factory decomposition becomes infeasible. Furthermore, centering efforts on the makespan-defining factory appears unsuitable due to the stronger interdependencies introduced by transportation.

1.1.3. Transportation efforts

Transportation has received limited attention in the literature related to the DPFSP. Looking at the role of the DPFSP within supply chain planning, transportation connects procurement, production, and distribution within a supply chain. These stages correspond to the transportation of jobs to the distributed factories before production, the transportation of intermediate jobs between factories, and the delivery of finished jobs to customers after production.

Several papers deal with the transportation after production within the DPFSP. Hou et al. (2022) extended the DPFSP to incorporate a Capacitated Vehicle Routing Problem after production, planning the distribution of jobs to customers from distributed flow shops. Fu et al. (2022) also focused transportation of finished jobs, integrating a multi-depot vehicle routing problem with the DPFSP to minimize the maximum delivery time for jobs to customers. Another paper by Li, Li, Zhang et al. (2021) tackled aggregate finished jobs to batches for delivery, considering makespan and energy consumption as objectives. In addition, Schulz et al. (2022) also investigate transportation of finished jobs, utilizing makespan and carbon emissions from both production and transportation as objective functions.

Regarding transportation efforts before production, several papers have extended the DPFSP to consider this case. Cai et al. (2017) consider transportation from suppliers to the distributed factories, accounting for three objective functions, i.e., makespan, maximum lateness, and total costs, which also includes transportation costs. Li, Li and Gao

(2021) deal with transportation to a set of heterogeneous flow shops minimizing makespan, modeling the time required for transportation by using factory-specific release dates.

Looking at transportation of intermediate jobs between factories, no study exists dealing with this type of transportation in the scope of distributed flow shop scheduling, possibly due to the complexities and challenges it presents. Recently, Han et al. (2023) consider the problem of two serial distributed permutation flow shop environments, where jobs have to be transported between flow shops after completing processing in the first and before processing in the second distributed permutation flow shop. However, this does not allow for inter-factory transportation between machines of a distributed flow shop. The latter type of transportation opens up a large number of transportation opportunities, increasing with the number of machines. In their review, Perez-Gonzalez and Framinan (2023) point out that this type of transportation represents a research gap for the DPFSP. We note that the permutation constraint should not be applied when investigating the DFSPT. Allowing jobs to switch between distributed flow shops, while maintaining a single permutation per flow shop, could potentially limit the quality of the solutions obtained.

1.2. Contribution

Our contribution is threefold. First, we introduce the Distributed Flow Shop Scheduling Problem With Inter-Factory Transportation (DFSPT). Unlike previous research, our work addresses the transportation of jobs between machines, which has not been considered in prior studies. Previous literature focused on job transportation either from a preceding facility to the first machine or to a customer after the job is finished on the last machine. The DFSPT fills this gap by incorporating the transportation between machines. This problem variant presents new challenges since the permutation constraint cannot be applied when jobs may switch between factories in-between machines, adding to the problem complexity. Furthermore, compared to problem settings with transportation which were previously considered, it enables a higher number of transportation opportunities which increases with the number of machines leading to an increased solution space.

Second, we propose a mathematical formulation, a constraint programming formulation, and a novel heuristic algorithm based on iterated greedy search that adapts state-of-the-art approaches to tackle the DFSPT. Our algorithm incorporates transportation modeling in a way that allows it to efficiently find high-quality solutions for the DFSPT. Based on a graph representation a problem-specific speed-up procedure is developed that further improves the search capabilities of the algorithm.

Finally, we validate the performance of our proposed algorithm through extensive computational experiments. Our results provide valuable insights into the impact of transportation on distributed flow shop scheduling. We use approximate upper and lower bounds for the DFSPT based on heuristics for the DPFSP and the Hybrid Flow Shop Scheduling Problem (HFSP) respectively, shedding light on the potential benefit of transportation. The quantification of the trade-offs between transportation efforts and makespan reduction is an essential information for decision makers in distributed manufacturing systems. Furthermore, we generate a comprehensive set of test instances to validate the effectiveness and efficiency of our heuristic approach in a comparison to the mathematical and constraint programming formulations.

1.3. Organization of this paper

The organization of this paper is as follows. Section 2 presents a formal description of the DFSPT, as well as a mathematical programming and constraint programming approach. Section 3 presents our solution approach for the DFSPT, based on an iterated greedy search procedure. We present our computational studies in Section 4 that demonstrate the efficiency of our algorithm and analyze the value of transportation in a distributed flow shop scheduling context. Finally, we conclude our paper in Section 5 and offer an outlook on avenues for further research.

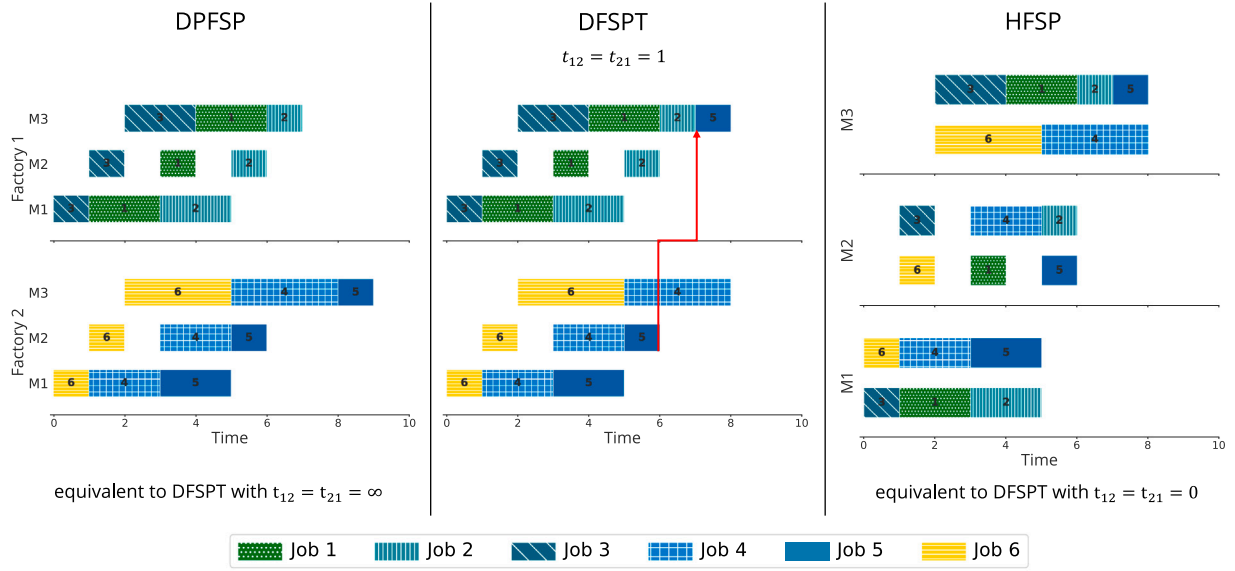


Fig. 2. Illustrative example of the DFSPT and its relation to the DPFSP and HFSP.

2. Problem description and mathematical formulations

2.1. Problem description

In the DFSPT, each factory ($k \in K$), consists of an identical series of machines ($i \in I$). Each job ($j \in J$) must be processed sequentially on these machines, whereas every factory has the capability of processing any given job. The jobs are characterized by their processing times, p_{ij} , on each machine. We assume that each machine can only process one job at a time, and each job can be processed only on one machine at a time. Furthermore, there are unlimited buffers between machines. When a job starts processing on a machine, it continues processing until finished, i.e., there are no preemptions. After a job finishes processing on a given machine, it is possible to transport this job to a different factory. The transportation times depend on the distance between the origin and destination factories and can potentially delay further processing of the jobs. In case of a transportation after machine i from factory k to k' , a transportation time of $t_{kk'}$ is incurred, after which the job may resume processing on machine $i + 1$ in factory k' . The aim is to find an assignment of jobs to the available factories and a sequence for the jobs on the machines, accounting for the transportation of jobs. In this, the objective is to minimize the makespan across all factories, denoted by $C_{\max}$.

Example 1. Table 1 shows the processing time data for an example to illustrate the concept of transportation in the context of the DFSP. We consider a problem instance with two factories, three machines, and a number of six jobs. Furthermore, we assume that one time unit is required for transportation between the two factories. Fig. 2 shows optimal schedules for the same problem instance for the DPFSP (left), the DFSPT (middle), and the HFSP (right). Without the option of transportation, the optimal makespan is 9, since job 5 has to wait for job 4 to complete before processing on machine 3. With the possibility of transportation, we can transport job 5 from factory 2 to factory 1 after finishing on machine 2. Then, job 1 incurs one time unit of transportation time and is available for processing in factory 1 at time 7. This enables a more efficient utilization of the resources of machine 3 in factory 1 and reduces the makespan to 8. If all machines are in the same factory, the problem corresponds to the HFSP such that all transportation times are zero. For our example, we obtain no further reduction of the makespan with zero transportation times.

Table 1

Processing time data for the example in Fig. 2.

M_i/J_j	J_1	J_2	J_3	J_4	J_5	J_6
M_1	2	2	1	2	2	1
M_2	1	1	1	2	1	1
M_3	2	1	2	3	1	3

Transportation time $t_{12} = t_{21} = 1$.

2.2. Mathematical programming formulation

This section presents a mathematical programming formulation (MP) for the DFSPT. Our mathematical formulation can be seen as an extension of the existing mathematical programming approach for the distributed flow shop scheduling problem (Naderi et al., 2023). Table 2 provides an overview of the notation used for our mathematical and constraint programming formulations.

To model the DFSPT, we introduce several decision variables. Variables C_{ij} represent the completion time of job j on machine i . Further, binary variables w_{kij} indicate whether job j is processed in factory k for machine i . To account for transportation, binary variables $z_{kk'ij}$ indicate if job j is transported from factory k to k' after completing its processing on machine i . Additionally, binary variables $x_{ijj'}$ capture precedence relationships between jobs.

$$\text{minimize } C_{\max} \quad (1)$$

$$C_{1j} \geq p_{1j} \quad \forall j \in J \quad (2)$$

$$C_{ij} \geq C_{(i-1)j} + p_{ij} + \sum_{k \in K} \sum_{k' \in K} t_{kk'} z_{kk'ij} \quad \forall i \in I \setminus \{1\}; j \in J \quad (3)$$

$$C_{\max} \geq C_{ij} \quad \forall i \in I; j \in J \quad (4)$$

$$\sum_{k \in K} w_{kij} = 1 \quad \forall i \in I; j \in J \quad (5)$$

$$w_{k(i+1)j} \geq w_{kij} - \sum_{k' \in K} (z_{kk'ij} + z_{k'kij}) \quad \forall k \in K; i \in \{1, \dots, m-1\}; j \in J \quad (6)$$

$$C_{ij} \geq C_{ij'} + p_{ij} - M(3 - x_{ijj'} - w_{kij} - w_{kij'}) \quad \forall i \in I; k \in K; j, j' \in J | j > j' \quad (7)$$

Table 2

Notation for mathematical programming and constraint programming formulations.

Index set	Description
$J = \{1, \dots, n\}$	Set of jobs
$I = \{1, \dots, m\}$	Set of machines
$K = \{1, \dots, f\}$	Set of factories
Parameters	Description
p_{ij}	Processing time of job j on machine i
M	Sufficiently large constant
$t_{kk'}$	Transportation time between factories
d	Transition matrix which denotes the transportation time between factories
Decision variables (MP)	Description
$C_{\max}$	Makespan
C_{ij}	Completion time of job j on machine i
$x_{ijj'}$	1 if job j' precedes job j on machine i ; 0 otherwise
w_{kij}	1 if factory k processes job j on machine i ; 0 otherwise
$z_{kk'ij}$	1 if job j is transported from factory k to factory k' after machine i ; 0 otherwise
Decision variables (CP)	Description
T_{ij}	Interval variable for the processing interval of job j on machine i
A_{kij}	Optional interval variable for the processing interval job j on machine i in factory k
MS_{ki}	Sequence variable for jobs on machine i of factory k
FS_j	Sequence variable for the factory sequence of job j

$$C_{ij'} \geq C_{ij} + p_{ij'} - M(2 + x_{ijj'} - w_{kij} - w_{k'ij'}) \quad \forall i \in I; k \in K; j, j' \in J | j > j' \quad (8)$$

$$C_{ij}, C_{\max} \geq 0, w_{kij}, z_{kk'ij}, x_{ijj'} \in \{0, 1\} \quad \forall i \in I; j, j' \in J; k, k' \in K \quad (9)$$

The objective function (1) minimizes the makespan across all factories. Constraints (2) initialize the completion time of each job on the first machine ($i = 1$) and set the starting point for calculating completion times on subsequent machines. Constraints (3) calculate the completion time of jobs on all other machines ($i > 1$) based on the completion time on the previous machine, the processing time of the job on the current machine, and the transportation time between factories. We account for the transportation time by summing the product of the transportation time between factories ($t_{kk'}$) and binary decision variable $z_{kk'ij}$, which indicates whether job j is transported from factory k to factory k' after machine i . This constraint ensures that the completion time on machine i includes the transportation time between factories in case a job changes factories. Constraints (4) capture the value of the makespan, $C_{\max}$, such that it is greater or equal to the maximum completion time across all jobs on all machines. Constraints (5) require that each job is assigned to exactly one factory for processing on each machine such that a valid assignment of jobs to factories is maintained. Further, Constraints (6) ensure continuity of factory assignment between consecutive machines and connect the factory assignment variables (w_{kij}) to the transportation variables ($z_{kk'ij}$). There are three cases: First, if a job is processed in the same factory on consecutive machines, there is no transportation ($w_{k(i+1)j} = w_{kij}$). Second, if a job is transported to factory k ($z_{k'kij} = 1$), then it was previously assigned to a different factory ($w_{kij} = 0$) and may be assigned to factory k from machine $i + 1$ on ($w_{k(i+1)j} = 1$). Inversely, if a job is transported from factory k to a different factory ($z_{kk'ij} = 1$), then it does not have to be assigned to factory k from machine $i + 1$ on ($w_{k(i+1)j} = 0$). Finally, Constraints (7) and (8) are precedence constraints, ensuring that no two jobs overlap on the same machine and in the same factory. These constraints only apply if jobs j and j' share machine i in the same factory k , i.e., $w_{kij} = w_{kij'} = 1$. Further, only one

of the constraints applies depending on the respective job precedence ($x_{ijj'}$). The constraints are only applied to jobs which are processed in the same factory for the respective machine. Constraints (9) state the domains of our decision variables.

2.3. Constraint programming formulation

In the following, we present our constraint programming formulation (CP) for the DFSPT. This formulation extends the formulation of Naderi et al. (2023) for the distributed flow shop scheduling problem and uses the constraint programming syntax of the IBM ILOG CP Optimizer.

$$\text{minimize } \left(\max_{j \in J} (\text{EndOf}(T_{mj})) \right) \quad (10)$$

$$T_{ij} = \text{IntervalVar}(p_{ij}) \quad \forall i \in I, \forall j \in J \quad (11)$$

$$A_{kij} = \text{IntervalVar}(p_{ij}) \quad \forall k \in K, \forall i \in I, \forall j \in J \quad (12)$$

$$\text{Alternative}(T_{ij}, A_{kij} : k \in K) \quad \forall i \in I, \forall j \in J \quad (13)$$

$$MS_{ki} = \text{SequenceVar}(A_{kij} : j \in J) \quad \forall k \in K, \forall i \in I \quad (14)$$

$$\text{NoOverlap}(MS_{ki}) \quad \forall k \in K, \forall i \in I \quad (15)$$

$$\text{EndBeforeStart}(T_{(i-1)j}, T_{ij}) \quad \forall i \in I \setminus \{1\}, \forall j \in J \quad (16)$$

$$FS_j = \text{SequenceVar}(A_{kij} : k \in K, i \in I) \quad \forall j \in J \quad (17)$$

$$\text{NoOverlap}(FS_j, d) \quad \forall j \in J \quad (18)$$

We define interval variables T_{ij} to model the start and end time of job j on machine i with duration p_{ij} (Exprs. (11)). Next, we introduce the optional interval variables A_{kij} to capture the factory-specific start and end times of jobs (Exprs. (12)). We introduce alternative constraints (Exprs. (13)) to connect variables T_{ij} and A_{kij} and ensure that each job is assigned to exactly one of the factories. Furthermore, we define sequence variables MS_{ki} on variables A_{kij} to model the sequence of jobs for every machine in each factory (Exprs. (14)). We impose no overlap constraints (Exprs. (15)) on variables MS_{ki} to prevent overlap of jobs on each machine and factory. To ensure that an operation i only starts on the next machine after completing the operation on machine $i - 1$, we impose end before start constraints on variables T_{ij} (Exprs. (16)). We introduce sequence variables FS_j (Exprs. (17)) to model the factory sequence of each job across machines. Then, Exprs. (18) ensure that transportation times are accounted for in case a job switches factory between machines. We ensure this by passing a transition matrix d with information on the transportation times between factories k and k' to the no overlap constraint on variables FS_j . Finally, the objective function (10) minimizes the makespan, which is the maximum completion time across all jobs on the last machine m .

2.4. Bounds on the value of transportation

In the context of the DFSPT, it is easy to see that the optimal makespan solution of the HFSP provides a lower bound on the objective value of the DFSPT. This holds true since transportation time can only increase the total makespan of jobs moving between factories. Therefore, $C_{\max}^{\text{DFSPT}*} \geq C_{\max}^{\text{HFSP}*}$. The HFSP can be viewed as a special case of the DFSPT, where the transportation time between all factories is 0. When the transportation time between factories is 0, the DFSPT reduces to the HFSP.

On the other hand, the DPFSP presents an upper bound on the objective value for the problem with transportation. Given an optimal solution to the DPFSP, we can transform it into a feasible solution for the DFSPT by allowing transportation between factories. By doing this, we relax the constraint that prevents jobs to switch between factories. Therefore, there is always a solution for the DFSPT with a makespan

lower or equal to the optimal solution of the DPFSP for the same problem instance. Consequently, $C_{max}^{DFSPT*} \leq C_{max}^{DPFSP*}$, i.e., C_{max}^{DPFSP} provides an upper bound on the objective value of C_{max}^{DFSPT} . Fig. 2 presents an illustrative example for the relationship between the DPFSP, the DFSPT, and the HFSP.

3. Solution approach

In this section, we describe our solution approach based on iterated greedy search. We use a full solution representation which explicitly accounts for the sequence of jobs on each machine in each factory to model transportation. Since evaluating all insertions, i.e., checking all factories and positions on machine i for an operation (i, j) as part of the local search, is typically the most computationally expensive part of a heuristic, we use a speed-up procedure to find the best insertion.

The speed-up procedure is based on the ideas proposed by Taillard (1990) and adapted for hybrid flow-shop scheduling by Fernandez-Viagas (2022). We extend the speed-up procedure for the DFSPT, maintaining the same complexity reduction of checking all insertions for an operation from $\mathcal{O}(n^2 \cdot m)$ to $\mathcal{O}(n \cdot m)$. We also provide a convenient graph representation which illustrates our speed-up procedure based on critical paths. Furthermore, we show how to obtain stronger bounds in case of transportation and adapt the procedure to an operation-based destruction neighborhood for full solution representations.

Section 3.1 presents our graph representation for solutions of the DFSPT. In Section 3.2, we describe the speed-up approach for the DFSPT. Next, Section 3.3 presents the iterated greedy search procedure.

3.1. Graph representation

In general, a job schedule for the DFSPT prescribes the assignment of jobs to factories for each machine, as well as the sequence of operations on each machine of each factory. We represent a solution for the DFSPT on a directed acyclic graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$. Let π_{ki} denote the ordered set of jobs being processed on machine i of factory k , corresponding to a particular solution of the DFSPT.

Fig. 3 illustrates the structure of $\mathcal{G}$ as a way to represent solutions of the DFSPT. In a feasible solution, the set of vertices includes nodes for each operation (i, j) as well as two additional nodes S and T , representing the source and sink of $\mathcal{G}$, respectively. All operation nodes have weights corresponding to the required processing time. The source has edges towards the first operation on the first machine of each factory k . Further, we connect the last operation on the last machine of each factory to the sink.

Every operation node has up to two incoming and outgoing edges. If an operation has a predecessor on the same machine, i.e., it is not the first job on the machine, there is an incoming edge from the preceding operation. Further, an operation has an outgoing edge to the next operation on the same machine except if it is the last operation. For machines $i > 1$, there is an incoming edge from the previous machine operation. If a job j is transported between factories k and l after machine i , the edge that connects operation (i, j) and operation $(i+1, j)$ is weighted with the transportation time t_{kl} . In graph $\mathcal{G}$, the critical path from node S to node T corresponds to the makespan of the schedule.

3.2. Speed-up procedure for evaluating insertions for the DFSPT

We illustrate the speed-up procedure for makespan computation in a DFSPT based on our graph representation. The speed-up procedure allows to efficiently determine the makespan of operation insertions into a partial schedule. Furthermore, our method accounts for the case of one or more unassigned operations to allow for an application as part of an operation-based insertion neighborhood. An unassigned operation (i, j) is not assigned to a factory k for machine i . We discuss how to strengthen the lower bound on the makespan in a partial schedule with one or more unassigned operations.

3.2.1. Determining the makespan of an insertion

We assume that some operation (i, j) has been removed from the schedule. Next, we seek to determine the makespan when inserting operation (i, j) after the operation in position h of machine i in factory k . We define $CP(n_1, n_2)$ as the length of the critical path from node n_1 to node n_2 in graph $\mathcal{G}$. Furthermore, let factory u denote the factory job j is assigned to on machine $i-1$ and let factory v be the factory job j will be processed in on machine $i+1$. For ease of notation, we further define the job preceding j on machine i as α_{ij} . We define $\alpha_{ij} = 0$ if j is the first job on machine i . Further, let σ_{ij} be the job which succeeds job j on machine i . If job j is the last job on machine i , we define σ_{ij} as $n+1$.

To evaluate an insertion of an unassigned operation (i, j) in all positions h and all factories k , we need to compute the critical path for all nodes in graph $\mathcal{G}$ which could be predecessors or successors on the same machine after the insertion. Furthermore, we need to compute the critical path for nodes $(i-1, j)$ and $(i+1, j)$. We determine the makespan after the insertion as follows:

$$C_{max} = \max \left\{ CP(S, (i-1, j)) + t_{uk}, \quad \text{(Previous machine)} \right. \quad (19)$$

$$\left. CP(S, (i, \alpha_{ij})) \right\} \quad \text{(Predecessor)} \quad (20)$$

$$+ p_{ij} \quad \text{(Processing time)} \quad (21)$$

$$+ \max \left\{ CP((i+1, j), T) + t_{kv}, \quad \text{(Next machine)} \right. \quad (22)$$

$$\left. CP((i, \sigma_{ij}), T) \right\} \quad \text{(Successor)} \quad (23)$$

Eqs. (19)–(23) determine the longest path from the source S to the sink T of graph $\mathcal{G}$ that passes through node (i, j) . The first two terms (Eqs. (19)–(20)) determine the longest path from the source to node (i, j) . The longest path can pass through the previous machine operation $(i-1, j)$ or the preceding operation on the same machine (i, α_{ij}) . If job j is inserted into a different factory on machine i , the first term (19) accounts for the transportation time which emanates from the previous machine operation to the insertion position. Term (21) is the processing time of operation (i, j) . The last two terms determine the longest path from the node (i, j) to the sink. Analogously, the longest path can pass through the next machine operation $(i+1, j)$ (22) or the successor on the same machine (23). When job j changes factories between machines i and $i+1$, we consider the weight of the resulting transportation arc (22).

The makespan after the insertion only considers the critical path from the source to operation (i, j) and from the inserted operation to the sink. If the makespan determined by Eqs. (19)–(23) is lower than the makespan of the partial schedule, we set the makespan after the insertion to the makespan of the partial schedule. Therefore, the makespan after the insertion of operation (i, j) in position h on machine i of factory k is the maximum of the makespan of the partial schedule and the makespan after the insertion. In this case, operation (i, j) is not part of the critical path $CP(S, T)$ of graph $\mathcal{G}$ after the insertion. However, the makespan is still determined by the critical path from the source to the sink.

In Section 3.2.3, we illustrate the calculation of the critical paths which requires only a single forward and backward pass through graph $\mathcal{G}$. Using this method for determining the makespan after an insertion, we only have to compute the forward and backward completion times for an operation once to evaluate all possible insertions. This greatly reduces the complexity of the local search process.

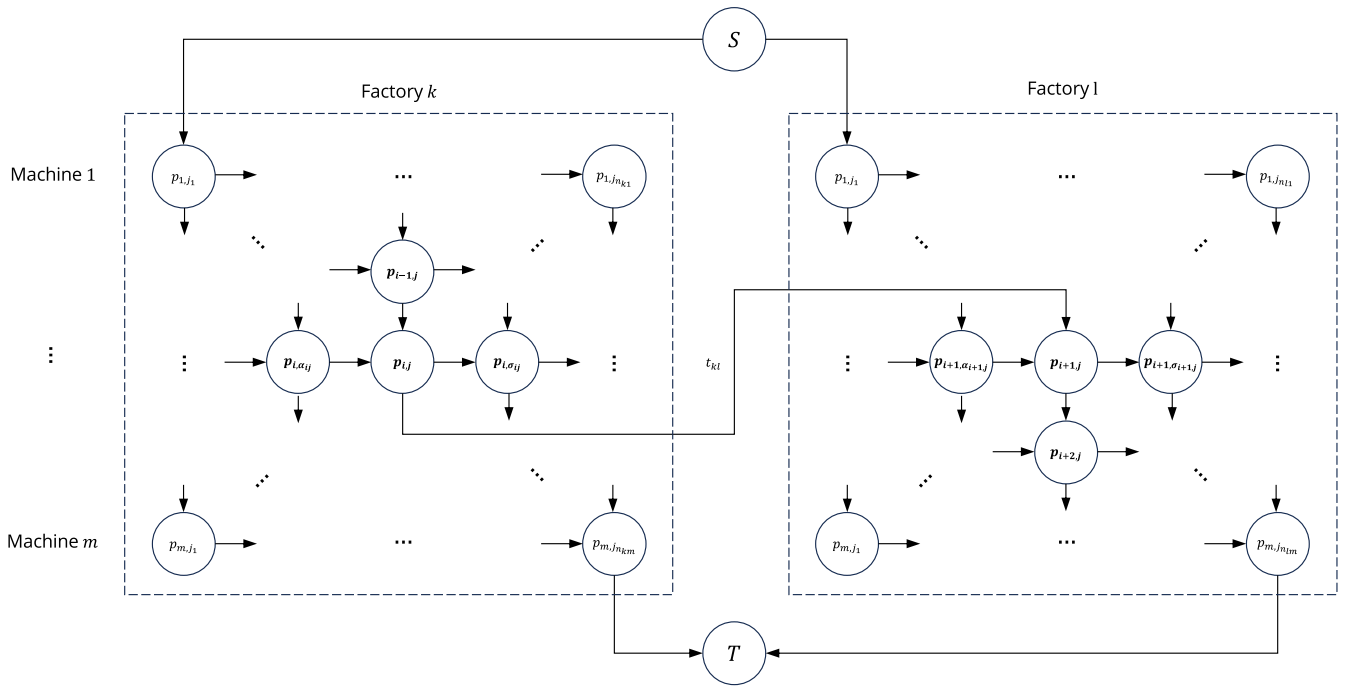


Fig. 3. Graph representation for a DFSPT solution.

Example 2. In the following, we provide an illustrative example for determining the makespan of an insertion based on our graph representation. Note that the example is based on the data from [Example 1](#) and illustrates the transition from the schedule on the left-hand side to the schedule in the middle of [Fig. 2](#). [Fig. 4](#) shows the corresponding graph representation for the exemplary schedule of [Fig. 2](#).

The schedule represented by [Fig. 4](#) has a makespan of 9. We now assume that operation (3,5) is dropped and evaluate an insertion in the last position on machine 3 of factory 1. We note that dropping operation (3,5) leads to a partial schedule with a makespan of 8. If dropping an operation does not reduce the makespan, there is no reinsertion that reduces the makespan.

[Fig. 5](#) illustrates the reinsertion of operation (3,5) in the last position of machine 3 of factory 1. We can now apply the procedure described in the previous section to determine the makespan of the insertion.

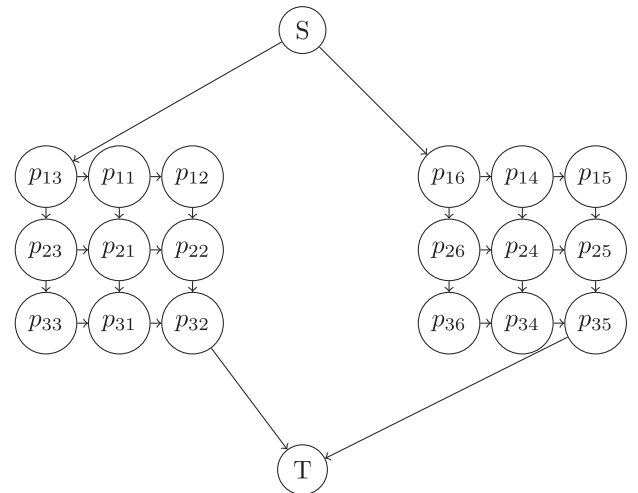
$$\max\{CP(S, (2, 5)) + 1, CP(S, (3, 2))\} + p_{35} = \max\{6 + 1, 7\} + 1 = 8 \quad (24)$$

As we insert operation (3,5) in the last position on the last machine, there is no succeeding operation for the next machine and no successor on the same machine. After the insertion, the solution has a makespan of 8 which reduced the makespan by one time unit compared to the original solution. It is easy to see that the solution is optimal for the DFSPT.

3.2.2. Improved lower bound on the makespan

Our heuristic framework relies on an operation-based neighborhood structure where partial schedules occur. When selecting the best position for an insertion of an operation into a partial schedule, we consider the impact of the insertion on the makespan. If an insertion of an unassigned operation is evaluated for a schedule with two or more unassigned operations, the resulting schedule after the insertion is a relaxation and the critical path $CP(S, T)$ of $\mathcal{G}$ for this schedule provides a lower bound on the makespan of a full schedule.

To improve convergence, we present two techniques that improve the lower bound on the makespan. First, when an operation (i, j) is unassigned, we insert a node into the graph that is not part of any factory, relaxing machine sequence constraints. We connect this node to the previous and next machine operations $(i - 1, j)$ and $(i + 1, j)$ to

Fig. 4. Graph representation of the solution of the exemplary DFSP schedule in [Fig. 2](#).

improve the makespan bound. In this way, we consider the processing time required after completing operation $(i - 1, j)$ before operation $(i + 1, j)$ can start. Second, we analyze the minimum transportation time of unassigned operations. We assume that job j is assigned to factory k for machine $i - 1$ and to factory l for machine $i + 1$. When operation (i, j) is unassigned, the minimum transportation time incurred after reassigning operation (i, j) is t_{kl} . When $k = l$, no inference in terms of the minimum transportation time is possible. For $k \neq l$, it is clear that for a transportation time matrix satisfying the triangular equality the minimum transportation time incurred after reinserting operation (i, j) is t_{kl} . We consider this lower bound as part of graph $\mathcal{G}$ by adding an augmented transportation arc with transportation time t_{kl} connecting node (i, j) to node $(i + 1, j)$.

We note that these bounds are also valid for the case where an operation is unassigned for multiple successive machines.

representation is not suitable for the DFSPT as we found that this solution representation could not achieve high quality solutions in many problem instances. Therefore, we use a full solution representation, as we explicitly keep a list with the sequence of jobs j on each machine i in each factory k . This allows for a detailed modeling of transportation options as part of the solution representation.

Our initialization procedure begins by applying the IG-DPFSP heuristic. This heuristic determines a job permutation for each factory, minimizing makespan. To adapt the solution from a permutation to a full representation solution for the DFSPT, we simply impose the same permutation ordering for all machines of each factory, as suggested by the DPFSP solution. It is easy to see that the full representation solution for the DFSPT possesses the same makespan as the permutation solution for the DPFSP, since no transportation occurs. IG-DPFSP provides state-of-the-art performance for the DPFSP which ensures that it provides a high-quality initial solution for the DFSPT.

3.3.2. Destruction procedure

The destruction operator partially destroys the current solution to diversify the local search (see Algorithm 2). The destruction operator destroys a number of d operations. In every iteration, we first establish a critical set C consisting of operations that are part of the critical path, i.e., define the makespan of the current solution. Next, we randomly select an operation from the critical set, denoted as (i, j) , and subsequently remove the operation from the current solution. We add every removed operation to set D to keep track of the destroyed operations. The destruction operation ultimately yields the set of destroyed operations D to be reinserted in the construction phase.

In our experiments, we also evaluate a job-based destruction and construction procedure which is used in many existing iterated greedy heuristics for flow shop scheduling (see, e.g., [Fernandez-Viagas, 2022](#); [Ruiz et al., 2019](#)). When applying the job-based destruction and construction procedures, all operations of the selected job are deleted and subsequently reinserted during construction.

Algorithm 2 Destruction operator

```

1: procedure DESTRUCTION( $d$ )
2:    $D \leftarrow \emptyset$ 
3:   for  $idx = 1 \rightarrow d$  do
4:      $C \leftarrow$  Critical set of current (partial) solution
5:     Select random operation  $(i, j)$  from  $C$ 
6:     Delete  $(i, j)$  from current solution
7:      $D \leftarrow D \cup \{(i, j)\}$ 
8:   end for
9:   return  $D$ 
10: end procedure

```

3.3.3. Construction procedure

The construction operator recovers feasibility by reinserting the operations removed by the destruction operator into the solution (Algorithm 3). For each operation $(i, j) \in D$, we determine the position and factory that lead to the minimum makespan when reinserting the operation. In determining the best position and factory, we use the speed-up procedure detailed in Section 3.2. After each reinsertion, we apply local search $LS1$, based on reinserting all operations of the critical set.

3.3.4. Local search 1

The local search procedure $LS1$ performs a local search on the given schedule by reinserting every operation on the critical path (Algorithm 4). In every iteration, we first determine the critical set. The critical set is then shuffled to diversify the search. Then, we remove and reinsert each operation of the critical set at the best possible position and factory. If there is a reinsertion which leads to a lower makespan

Algorithm 3 Construction operator

```

1: procedure CONSTRUCTION( $D$ )
2:   for each  $(i, j)$  in  $D$  do
3:      $(p, k) \leftarrow$  Determine best insertion for operation  $(i, j)$  using the
       speed-up procedure
4:     Insert job  $j$  at position  $p$  on machine  $i$  in factory  $k$ 
5:     Apply  $LS1$  to improve the current (partial) solution
6:   end for
7: end procedure

```

compared to the makespan with the previous position and factory, we place the operation at the new position. Subsequently, we recalculate the critical set and restart the local search procedure. Otherwise, we reinsert the operation at the previous position.

If we are unable to reinsert any of the operations of the critical set into a position that leads to a reduced makespan, the local search procedure terminates.

Algorithm 4 $LS1$

```

1: procedure  $LS1(s)$ 
2:    $crit\_set \leftarrow$  Identify the critical operations in  $s$ 
3:    $improve = true$ 
4:   while  $improve$  do
5:      $improve = false$ 
6:     for each operation in  $crit\_set$  do
7:        $s' \leftarrow$  Create a copy of  $s$ 
8:       Remove operation from  $s'$ 
9:       Reinsert operation into best position in  $s'$ 
10:      if  $s'$  has shorter makespan than  $s$  then
11:         $s \leftarrow s'$ 
12:         $crit\_set \leftarrow$  Update the critical operations in  $s$ 
13:         $improve = true$ 
14:      break
15:    end if
16:  end for
17:  end while
18:  return  $s$ 
19: end procedure

```

3.3.5. Local search 2

Our second local search procedure ($LS2$) is based on a neighborhood where we reinsert all operations of a particular job (Algorithm 5). We evaluate all jobs in terms of whether their removal and reinsertion leads to a reduced makespan. Therefore, $LS2$ is more computationally expensive and only applied once at the end of each iteration of our heuristic.

In each iteration of $LS2$, we start by generating a random permutation of jobs. Then, we iterate the jobs using the random permutation. For each job, we delete all operations and subsequently reinsert the operations into their best possible positions. If the makespan of the schedule is reduced after the reinsertion, we replace the incumbent schedule by the improved schedule and restart the iteration of jobs using the improved schedule. The search terminates when none of the jobs of schedule s can be reinserted with an improved makespan.

3.3.6. Acceptance criterion

We use the classical simulated annealing criterion proposed by [Ruiz and Stützle \(2007\)](#) to decide whether a solution replaces the incumbent. This acceptance criterion has been widely adopted in the context of iterated greedy algorithms for flow shop scheduling problems. Algorithm 6 describes this acceptance criterion. According to [Ruiz and Stützle \(2007\)](#), the temperature parameter is defined as $Temperature = T \cdot \frac{\sum_{i \in I} \sum_{j \in J} p_{ij}}{n \cdot m \cdot 10}$, where T is a hyperparameter.

Algorithm 5 LS2

```

1: procedure LS2( $s$ )
2:    $improve = true$ 
3:   while  $improve$  do
4:      $improve = false$ 
5:      $rnd\_job\_sequence \leftarrow$  Random permutation of jobs
6:     for each  $j$  in  $rnd\_job\_sequence$  do
7:        $s' \leftarrow$  Create a copy of  $s$ 
8:       Remove  $j$  from all machines in  $s'$ 
9:       for  $i \in \{1, \dots, m\}$  do
10:        Reinsert operation  $(i, j)$  into best position in  $s'$ 
11:       end for
12:       if  $s'$  has shorter makespan than  $s$  then
13:          $s \leftarrow s'$ 
14:          $improve = true$ 
15:         break
16:       end if
17:     end for
18:   end while
19:   return  $s$ 
20: end procedure

```

As a stochastic acceptance criterion, simulated annealing allows for the acceptance of solutions with a higher makespan than the current incumbent to avoid stagnation of the search in local optima. The probability increases with a lower difference between the makespan of the candidate and incumbent solution.

Algorithm 6 AcceptanceCriterion

```

1: procedure ACCEPTANCECRITERION( $s, s'$ )
2:    $C_{max}, C'_{max} \leftarrow$  Makespan of solutions  $s, s'$ 
3:    $random \leftarrow$  Uniform random number
4:   if  $random \leq \exp(-(C'_{max} - C_{max})/Temperature)$  then
5:     return true
6:   else
7:     return false
8:   end if
9: end procedure

```

4. Computational study**4.1. Experimental setting**

The aim of our computational study is twofold. First, we evaluate the performance of our proposed algorithm for the DFSPT. Second, we aim to gain insights into the magnitude of the value of transportation, i.e., the attainable reduction of makespan when the option of transportation is introduced in the context of distributed flow shop scheduling.

To gain insights into the value of transportation, two existing heuristics were reimplemented to serve as a basis for comparison and to determine lower and upper bounds on the makespan objective of the distributed flow shop scheduling problem with inter-factory transportation, as described in Section 2.4. For the DPFSP, we selected the algorithm by Ruiz et al. (2019) (referred to as IG-DPFSP). Additionally, we implemented the recent benchmark for the HFSP proposed by Fernandez-Viagas (2022) (referred to as IG-HFSP). We have implemented all algorithms in C++ to ensure efficiency and comparability. We validated our reimplementations by running the IG-DPFSP and IG-HFSP on the test sets used in their original implementation with the same time budget and a single CPU core. These preliminary experiments were conducted on a MacBook with a M1 Pro processor with 3.2 GHz clock speed and 16 GB RAM. Our implementation obtained lower makespan values on average across these experiments, identifying new lower bounds for many instances. We note that the original

studies used different hardware such that no clear comparison is possible. However, the results of our preliminary experiments indicate that the performance of our reimplementations is at least comparable to the implementation in the original paper.

All experiments reported as part of the computational studies were performed on the TU Dresden High Performance Computing cluster, specifically on the Intel Haswell partition (Intel Xeon E5-2680 v3 2.5 GHz), using 16 CPU cores with 32 GB of RAM to ensure a consistent environment across our tests. We parallelized all algorithms using OpenMP (Chandra et al., 2001). Our parallelization strategy is the same for all algorithms such that iterations of the iterated greedy search procedures are distributed across threads.

In our experiments, we use a small-scale and large-scale set of benchmark instances which we selected from the set of instances provided by Naderi and Ruiz (2010) which are based on the instance generator proposed by Taillard (1993). All benchmark instances use a uniform random distribution for job processing times such that processing times vary randomly from 1 to 99. For each combination of the number of job and machine parameters, the benchmark set provides ten random instances. We extend each problem instance by the transportation time values, assuming homogeneous transportation times between facilities. This approach is consistent with the approach of Naderi and Ruiz (2010) who similarly extend each problem instance by the numbers of factories. While our approach is amenable to heterogeneous transportation times, we focus on homogeneous transportation times to analyze the impact of varying transportation times on schedules without additional variability of different transportation times between facilities.

Specifically, for the small-scale problem set, we selected all problem instances with $f \in \{2, 4\}$ from Naderi and Ruiz (2010). Their small-scale problem set contains instances with 4 to 16 jobs and 2 to 5 machines. Further, we use two transportation times, i.e., 5 and 30. In total, the small-scale benchmark set consists of 560 problem instances of the DFSPT.

For our large-scale benchmark set, we selected the instance sets with 20 and 50 jobs, 5, 10, and 20 machines, and 2, 4, and 6 factories. Furthermore, we consider five transportation time values, i.e., 5, 10, 30, 50, and 100. We selected the range of transportation times based on the processing time distribution and aim to capture multiple settings ranging from several production facilities sharing the same premises to distributed factories with a larger distance in between. This results in a total of 900 problem instances for the DFSPT. For the DPFSP and HFSP, which do not consider transportation times, there are 180 unique instances in the large-scale benchmark set.

Our computational experiments consist of three parts. First, we conduct a hyperparameter experiment for IG-TP in Section 4.2. Second, we evaluate the performance of the MP and CP formulations against one another, as well as in comparison to the IG-TP heuristic (Section 4.3.2).

Specifically, we employed the mathematical programming solver CPLEX 22.1 and the constraint programming solver CP Optimizer 22.1. Third, we investigate the value of transportation based on the large-scale benchmark set by using the IG-DPFSP and IG-HFSP to determine lower and upper bounds (Section 4.4). As pointed out in Section 2.4, the optimal solutions for the DPFSP and the HFSP provide theoretical upper and lower bounds for the DFSPT. For $m \geq 3$, the flow shop scheduling problem minimizing makespan is NP-complete (Garey et al., 1976). Therefore, obtaining optimal solutions for these related problems is computationally hard within a reasonable time even for many small-scale instances. Therefore, we use the solutions provided by the heuristics as approximations for the lower and upper bounds. As these heuristics have been carefully designed and refined to provide high-quality solutions for the DPFSP and HFSP, and we run them with a high time limit, they are expected to provide high-quality solutions and can serve as reasonable proxies on the true values of the lower and upper bound. Finally, Section 4.4 also discusses the impact of transports on

schedules and the implications of our experiments for the option of transportation as part of distributed flow shops.

4.2. Hyperparameter tuning

To investigate the impact of our algorithm's hyperparameters on performance, we conducted a computational study on a subset of our problem instances. Specifically, we randomly picked a subset of 18 problem instances from the large-scale benchmark set such that the subset contains one problem instance for each combination of n , m , and f . Furthermore, we evaluated these problem instances for all five transportation times, obtaining a problem set with 90 instances. We deployed these experiments on the same servers with a number of 16 CPU cores each, limiting the run time to 600 s to keep the total required computational resources in a reasonable range.

In the following, we list the hyperparameters of our algorithm and describe the range of values that were tested:

- Destruction intensity: We evaluate several numbers of destroyed operations $d \in \{10, 20, 30\}$. Furthermore, we evaluate destroying 2, 4, or 6 jobs for the job-based operator, based on values typically applied in the literature (see, e.g., Fernandez-Viagas, 2022; Ruiz et al., 2019).
- T : We evaluate values $T \in \{0.1, 0.3, 0.5\}$ (see, e.g., Ruiz et al., 2019; Ruiz & Stützle, 2007).

We excluded destruction operators that destroy all operations on a particular machine or in a specific factory based on their performance in preliminary experiments. Further, we evaluated the inclusion of random perturbations and one-step lookahead local search. We note that the prior did not improve the solution quality and the computational time requirements outweighed the benefits of the latter. Therefore we focused on the experimental design above.

We evaluate two algorithm variants: the IG-TP algorithm with an operation-based destruction operator and the IG-TP algorithm with a job-based destruction operator which has been used in many flow shop scheduling heuristics. For each algorithm variant, we conduct a separate hyper-parameter experiment with a full factorial experimental design which results in 18 potential algorithmic configuration and 1620 algorithm runs for a total computational time of 0.49 CPU years. We measure the algorithmic performance in terms of the average relative percentage deviation (ARPD) which is defined as follows:

$$ARPD = \sum_{r \in \{1, \dots, R\}} \left(\frac{C_{max}^{eval} - C_{max}^{ref}}{C_{max}^{ref}} \cdot 100 \right) / R \quad (31)$$

C_{max}^{eval} is the makespan of the algorithm that we evaluate. We evaluate the performance with respect to a reference algorithm or reference result for the makespan, denoted as C_{max}^{ref} . R denotes the number of algorithmic results included in the computation of the ARPD. For the hyperparameter experiments, we use the best found solution for each problem instance across all algorithm runs during these experiments as C_{max}^{ref} , as there are no previous makespan results for DFSPT instances. Fig. 6 shows the results of our hyperparameter study using means plots for the average relative percentage deviation (ARPD) of each hyperparameter. We used Tukey's Honest Significant Differences (HSD) test to evaluate the statistical significance of the differences in performance between our tested hyperparameters. Tukey's HSD is a post-hoc test based on the analysis of variance (ANOVA), controlling for the Family-Wise Error Rate (FWER). Therefore, it allows for a comparison of hyperparameters with more than two values while controlling for the possibility of Type I errors that arises when making multiple comparisons (Midway et al., 2020). For each hyperparameter value, the means plots show the 95%-confidence interval based on Tukey's Honest Significant Differences. Thus, if the confidence intervals of two hyperparameter values within a means plot do not overlap, there is a significant difference between the performance in terms of

ARPD between the hyperparameters. Additionally, we conducted three ANOVA tests to analyze the impact of the hyperparameters on the ARPD of each of the two algorithm variants and to test whether there is a significant difference of the ARPD depending on the algorithm variant.

From the results of the ANOVA, it is evident that all hyperparameters have a significant impact on the algorithmic performance (see Tables 5–6 in the Appendix). In case of the operation-based destruction operator, it turns out that setting the number of destroyed operations to $d = 10$ leads to a significantly worse performance compared to values of $d = 20$ or $d = 30$. Therefore, it appears that destroying a higher number of operations is beneficial for the diversification of the search. While the mean is slightly lower for $d = 20$ than $d = 30$, the difference is not statistically significant. Looking at the job-based destruction operator, it turns out that destroying two jobs is significantly better than destroying six jobs. The difference between destroying two and four or four and six jobs is not significant.

When using the job-based destruction operator, the results indicate that a lower value for parameter T is significantly better. Parameter T is part of the simulated annealing acceptance criterion, whereas a lower value leads to an intensification of the search. Setting parameter T to a value of 0.1 or 0.3 is significantly better than using a value of 0.5. The difference between the values 0.1 and 0.3 is not statistically significant. Interestingly, a higher value for parameter T is better for the operation-based destruction operator such that using a value of $T = 0.3$ is significantly better than using a value of $T = 0.1$. Here, the difference between a value of $T = 0.1$ and $T = 0.5$ is not statistically significant.

Across the entire set of hyperparameter experiments, the ARPD is 1.13% when using the job-based destruction operator compared to an ARPD of 0.59% when using the operation-based destruction operator. The ANOVA shows that there is a significant difference between the mean ARPD of the job-based and operation-based variant of the IG-TP (see Table 7 in the Appendix). This indicates the advantageousness of an operation-based destruction neighborhood for the DFSPT compared to the job-based destruction neighborhood which has been the most popular neighborhood for the DPFSP and many extensions. Regarding the hyperparameters, the best combination for the job-based destruction operator is $T = 0.1$ and $d = 2$, resulting in an ARPD of 0.82%. For the operation-based destruction operator, the best parameter combination is $T = 0.3$ and $d = 20$ with an ARPD of 0.50%. In conclusion, we use the IG-TP with an operation-based destruction operator and set the hyperparameters as follows for our further experiments: We set $d = 20$, as it yields the lowest mean ARPD. Furthermore, we set parameter $T = 0.3$.

4.3. Computational performance of the exact and heuristic methods on the DFSPT

4.3.1. Evaluation of the MP and CP formulation on small-scale instances

Owing to the computational challenge of the DFSPT, we first apply the MP and CP formulation to the small-scale benchmark set to gain insight into the performance. We set a time limit of 600 s for the set of small-scale problem instances.

Table 3 shows the computational results of the mathematical and constraint programming formulation across the set of small-scale instances. The results show that solvers were able to solve many small-scale instances to optimality. However, the performance of both the MP and CP formulation differs clearly for instances with 4 to 8 jobs and instances with 10 to 16 jobs.

Up to 8 jobs, the CP formulation solves all instances with a low run time and the MP formulation only has a single unsolved instance, indicating that the DFSPT can be solved well by exact approaches for problem instances of this size. For instances with 10 or more jobs, there are many unsolved instances for both approaches which exceed 50% for some instances.



Fig. 6. Means plots for the results of the hyperparameter experiments (with 95% confidence intervals based on Tukey's HSD), subdivided by the two algorithm variants, i.e., the operation-based destruction operator and the job-based destruction operator. The y-axis shows the mean average relative percentage deviation for instances using the hyperparameter value.

Table 3

Computational results for the mathematical and constraint programming formulations on small-scale instances. *Solved* specifies the number of instances without an optimality gap. *Best* specifies the number of instances where the formulation identified a solution with a strictly lower makespan compared to the other one, respectively.

<i>n</i>	<i>m</i>	Mathematical programming				Constraint programming				No.
		Avg time	Avg gap (%)	Solved	Best	Avg time	Avg gap (%)	Solved	Best	
4–8	2–3	0.57	0.00	120	0	0.09	0.00	120	0	120
4–8	4–5	6.34	0.02	119	0	0.21	0.00	120	0	120
10–16	2–3	390.65	10.99	63	0	84.86	3.83	143	40	160
10–16	4–5	476.54	10.92	40	1	254.30	10.55	97	104	160

The MP approach remains with 218 out of 560 instances unsolved. The MP approach fails to solve many instances with only two machines, a scenario where the CP formulation performed better. No clear relationship was observed between the number of factories, distances, and the problem difficulty.

For the CP approach, 80 out of the 560 problem instances remain unsolved. The unsolved instances show a relatively high optimality gap which yields little information. Notably, unsolved instances exist even for relatively small problems with 10 jobs. Specifically, for instances with 16 jobs, 50 out of 80 remain unsolved. While all instances with two machines were solved, those involving 3, 4, or 5 machines present more difficulty, falling among the unsolved cases. As with the MP approach, no definitive link between problem parameters such as factories and distances and the difficulty was observed.

Overall, the CP approach clearly outperforms the MP formulation. CP not only solved a greater number of instances to optimality but also did so in less time. Furthermore, CP found strictly lower values for the makespan for 144 problem instances, where the MP approach only identified a lower makespan value than the CP approach in one case. On average, the CP approach provides a 0.53% lower makespan across all instances. This only represents a moderate improvement in solution quality, suggesting that both the MP and CP approach identify reasonable solutions for instances of this size. The optimality gaps appear to be linked to the challenge of obtaining tight bounds for the DFSPT.

Generally, the small-scale instances underscore the inherent difficulty of the problem, with even scenarios involving only 10 jobs proving challenging for exact methods. The number of jobs and machines has the most pronounced effect on running times, while the influence of factories and distances is less clear.

We have also applied the IG-TP to the set of small-scale instances with the same time limit. IG-TP always found a solution with a makespan that is at least as good as the makespan of the MP formulation. For 4 out of 560 instances, the CP formulation obtained a lower makespan than IG-TP while there were 54 instances where IG-TP identified a schedule with a lower makespan compared to the CP formulation. In the next section, we present a detailed evaluation of the performance of IG-TP compared to the MP and CP formulation on the large-scale benchmark instances.

4.3.2. Performance of the IG-TP heuristic

Table 4 evaluates the performance of the IG-TP heuristic by comparing it to the computational results for the MP and CP formulation for the large-scale benchmark set in terms of the ARPD. To compute the ARPD, we use the makespan obtained by the IG-TP heuristic as C_{max}^{ref} . Both the MP and CP formulation and the heuristic were executed under the same computational resources and were subject to an identical time limit of 1800 s across all problem instances.

The results exhibit a poor performance for the mathematical-programming-based formulation. The MP formulation only solves few instances to optimality and our heuristic IG-TP always finds a solution that is at least as good or better compared to the solution of the MP formulation. Furthermore, the MP formulation is unable to provide tight bounds on the makespan for most instance size categories, as evidenced by the high solution gaps. The ARPD across all instances of the MP formulation compared to the IG-TP heuristic is 53.74%. It is evident that the ARPD increases with the number of jobs and machines. This indicates that the MP formulation does not scale to larger-sized instances as well as the IG-TP heuristic. For the largest instance size category, with 50 jobs, 20 machines, and 6 factories, there is an ARPD of 497.43%. We note that the total ARPD is heavily influenced by some of the larger instance categories, specifically, instances with 50 jobs and

Table 4

Overview of the computational results for the constraint programming formulation and comparison with the results of the IG-TP heuristic.

n	m	f	CPLEX CP Optimizer			CPLEX MP Optimizer			IG-TP			
			No.	Opt.	Gap (%)	No.	Opt.	Gap (%)	ARPD CP	No. $C_{max}^{IG-TP} \leq C_{max}^{CP}$	ARPD MP	No. $C_{max}^{IG-TP} \leq C_{max}^{MP}$
20	5	2	50	0	47.99%	50	0	42.16%	0.91%	50	8.35%	50
		4	50	0	20.78%	50	0	25.07%	0.80%	44	6.69%	50
		6	50	12	6.41%	50	8	9.72%	−0.05%	32	3.48%	50
20	10	2	50	0	34.59%	50	0	36.30%	2.26%	48	9.96%	50
		4	50	0	11.12%	50	0	16.35%	1.18%	45	7.50%	50
		6	50	26	1.72%	50	8	5.28%	−0.13%	34	3.72%	50
20	20	2	50	0	24.29%	45	0	29.86%	2.10%	50	10.19%	50
		4	50	4	5.73%	50	0	11.86%	0.46%	36	7.52%	50
		6	50	38	0.35%	50	2	3.88%	−0.24%	32	3.49%	50
50	5	2	50	0	72.68%	50	0	75.86%	0.71%	50	13.94%	50
		4	50	0	51.53%	50	0	58.58%	1.68%	49	18.77%	50
		6	50	0	35.31%	50	0	45.90%	1.29%	44	20.97%	50
50	10	2	50	0	60.26%	50	0	68.58%	4.85%	50	31.65%	50
		4	50	0	37.46%	50	0	48.88%	4.39%	50	26.78%	50
		6	50	0	22.23%	50	0	36.05%	3.16%	50	24.83%	50
50	20	2	50	0	49.42%	50	0	61.91%	6.14%	50	39.74%	50
		4	50	0	27.75%	50	0	74.83%	4.62%	50	232.27%	50
		6	50	0	15.75%	50	0	76.54%	3.91%	50	497.43%	50
Total			900	80	29.19%	900	18	40.42%	2.11%	814	53.74%	900

20 machines. However, the MP formulation still exhibits high ARPD values even for smaller-sized instances. Overall, the IG-TP heuristic clearly outperforms the MP formulation across the problem set.

Surprisingly, the CP formulation greatly outperforms the MP formulation even more clearly than for the small-scale benchmark set. Overall, the CP formulation is also only able to solve few instances to optimality. Yet, the number of optimal solutions found is higher compared to the MP formulation. Most instances where the CP formulation can prove optimality originate from the category with 20 jobs and 6 factories, as is the case for the MP formulation. This could be attributed to the fewer number of jobs per factory in this particular case, thus limiting the solution space. For most problem instances, high gaps persist for the CP formulation as well, due to the weak solution bounds of the CP formulation which are of a greater magnitude for instances from the category with 50 jobs. While gaps appear to decrease with more factories, this is related to the distribution of jobs across factories which allows for improved job-based bounds on the makespan. Looking at the ARPD, the CP formulation significantly outperforms the MP formulation. The ARPD compared to the IG-TP heuristic is 53.74% for the MP formulation while the CP formulation obtains an ARPD of 2.11% across the test set. For instances with 50 jobs, the ARPD of the CP formulation increases to 3.42%, indicating an increased ARPD for larger problem instances. Overall, the performance of the CP formulation can be deemed reasonable given its ease of implementation in comparison to our heuristic algorithm IG-TP.

Looking at the results for the IG-TP heuristic, the heuristic clearly outperforms the MP formulation while presenting an improvement over the CP formulation for most instance size categories. Even though the CP formulation yields slightly better results for instances with 20 jobs and 6 factories where it can solve many instances optimally, the heuristic outperforms the CP formulation across all other instance categories. Interestingly, the heuristic manages to discover many of the same solutions in terms of makespan as the CP formulation for the 20 jobs and 6 factories category. Furthermore, the heuristic clearly scales better to larger-sized instances with 50 jobs, where the ARPD of the makespan found by the heuristic compared to the solver is larger than for the small-sized instances. This illustrates the superior performance of the heuristic on larger-sized instances. Specifically, for problem instances with 50 jobs, the heuristic finds solutions which improve upon the solution of the CP formulation for most problem instances. Furthermore, to realize the value of transportation, finding

high-quality solutions is crucial. We note that an improvement in the order of several percentage points may represent a large share of the value of transportation for many problem instances, as our detailed analysis of the value of transportation in the next section shows.

The results therefore illustrate the effectiveness and robustness of the IG-TP heuristic in finding high-quality solutions for the DFSPT. As an exact solution method, the CP formulation is clearly preferable to the MP formulation for the DFSPT. However, the results show that only few instances can be solved to optimality and both the MP and CP formulation obtain weak solution bounds.

4.4. Impact of transportation

In this section, we analyze the impact of transportation on the makespan and schedule based on the large-scale benchmark set. As discussed in the experimental setting, we solve the DPFSP and HFSP to gain a deeper understanding of the potential value of transportation.

We compare the solutions obtained by the IG-DPFSP for the DPFSP, the IG-TP for the DFSPT, and the IG-HFSP for HFSP. All heuristics use the same time limit of 1800 s. The makespan of the DPFSP provides an approximate upper bound on the makespan of the other solutions. Therefore, we standardize the makespan of the DFSPT and the HFSP using the makespan of the DPFSP. Fig. 7 provides an overview of the average standardized makespan for the DPFSP, the results for the DFSPT varied by transportation time, and the results for the HFSP. We define the value of transportation (VOT) as the relative percentage deviation of the makespan of the DFSPT from the makespan of the DPFSP, i.e., $VOT = \left(\frac{C_{max}^{DPFSP} - C_{max}^{DFSPT}}{C_{max}^{DPFSP}} \right) \cdot 100$. The VOT expresses the

possible reduction of the makespan when the option of transportation is introduced to the DPFSP. The results are grouped by the number of jobs, machines, and factories. For example, for the instance category with 20 jobs, 20 machines, and 2 factories, the makespan could be reduced by 7% on average when the transportation time is 0 (by comparing the value of 1.0 for the DPFSP to the value of the HFSP of 0.93). Looking at the range for the VOT, it is evident that the maximum potential for reducing the makespan across instance categories is obtained for 50 jobs, 20 machines, and 6 factories, approaching 11% (see HFSP values in Fig. 7).

For the largest transportation time of 100 time units, there is no improvement of the DPFSP makespan for most instance categories.

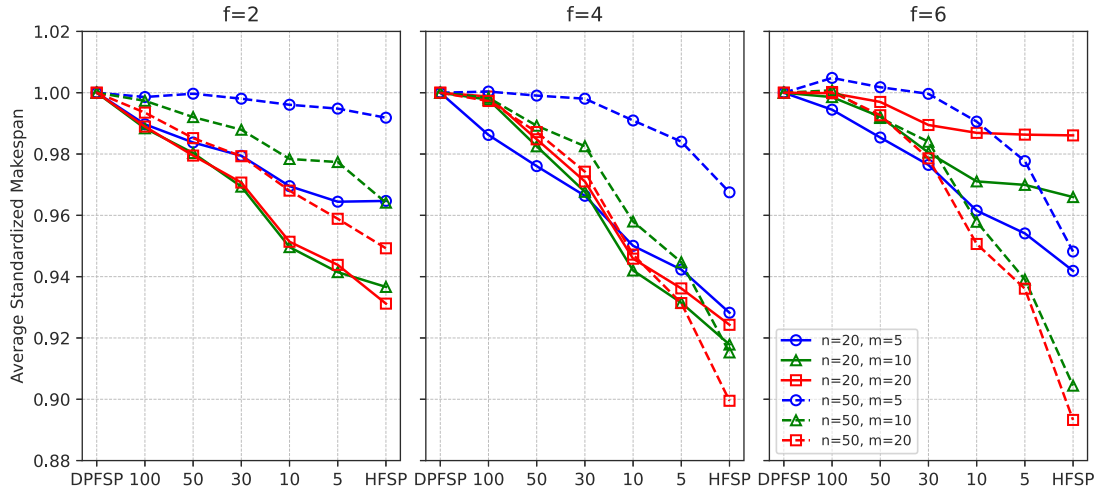


Fig. 7. Comparison of average makespan obtained by the IG-DPFSP, the IG-TP, and the IG-HFSP. All average makespans were divided by the average makespan obtained by the IG-DPFSP. Numbers on the x-axis indicate the solution of the DFSPT for the respective transportation time.

The values which slightly exceed 1.0 (see $f = 6$) indicate that the IG-TP solution provides a higher makespan than the DPFSP solution. The IG-TP heuristic only uses a small share of its run time applying the IG-DPFSP during initialization and then continues its search on a full solution representation. Therefore, in few cases the DPFSP solution outperforms the DFSPT solution in cases where the benefit of transportation is limited since applying the IG-DPFSP for the complete run time is more efficient. Except for the instance category with 50 jobs, 5 machines, and 2 factories, lower transportation times lead to a reduced makespan in all instance categories, i.e., there is a rising VOT with decreasing transportation times. However, the size of the reduction varies by instance category.

Realizing a larger fraction of the potential VOT appears to be mostly possible at very low transportation times. With a transportation time of 5, the average VOT across all instances is 4.37%, whereas the maximum VOT is 10.91%. In these cases, we observe that the DFSPT solution approaches the HFSP solution for most instance categories.

Fig. 8 shows box plots of the ARPD between the DPFSP and the DFSPT solutions, grouped by instance characteristics. In general, the box plots indicate a large spread of the VOT. The average VOT is clearly decreasing with an increase in transportation time between factories. The spread of the upper and lower quartiles of the VOT tends to decrease for higher transportation times, highlighting the higher potential of transportation for lower transportation times. At the same time, there are instances where transportation appears to have little benefit as well as other instances with a larger VOT in all instance categories. On average, the VOT is low for a transportation time of 100. However, Fig. 8 shows outliers in these instance categories where transportation is beneficial, with a VOT of up to 4.55%.

Looking at the impact of the number of jobs, it appears that the makespan reduction with transportation is higher for 20 jobs than for 50 jobs. This could be related to the possibility of attaining a better utilization of resources without transportation when distributing 50 jobs across factories compared to distributing 20 jobs. The makespan reduction when transportation is possible also appears to be larger for 10 or 20 machines compared to 5 machines. Based on the box plot for the number of factories, there is no clear influence on the makespan reduction.

Subsequently, we explore the number of transports included in the schedule of the DFSPT. The number of transports counts the number of times that any of the jobs is transported between factories. If a job is transported several times it counts towards the number of transports

multiple times. Generally, a schedule with fewer transport moves is more desirable if it maintains the same makespan. However, we do not consider the number of transports as an objective function. Therefore, we apply an automated post-processing step to remove unnecessary transport moves which do not impact the makespan in the following two cases. First, we remove transports if moving a single operation to a different factory reduces the number of transports. Second, we remove transports if an exchange of two operations of the same machine but of different factories reduces the number of transports.

Fig. 9 shows the number of transports observed on average for each instance category. Looking at the figure, we observe substantial variations in the number of transports across different scenarios. This variation seems to be tightly linked with the maximum number of transports. It is important to note that the maximum number of transports varies based on the number of jobs and machines, i.e., there are $n \cdot (m-1)$ possible transports. Therefore, the maximum number of transports is an order of magnitude higher in the largest vs. smallest instance category.

As expected, the transportation time has a high influence on the number of transports. The number of transports strictly decreases across all instance categories, when the transportation time rises. While some instances transport several hundred operations for a transportation time of 5, there are few transports when the transportation time is 100. This suggests that transports become less viable as the required time increases. Furthermore, when comparing the results of Figs. 7 and 9, the number of transports seems to be related to the VOT for the problem instance. Schedules with a higher number of transports tend to be associated with a higher VOT.

Across most instance categories, the number of transports increases when more factories are available. When six factories are available, instances with 20 jobs use a lower number of transports. The high number of available factories seems to facilitate the distribution of workloads in these instance categories such that a lower number of transports is required.

Overall, our results show that the possibility of transporting intermediate goods between factories in a distributed flow shop environment allows for an improved machine utilization in many cases, leading to reduced makespans. The VOT varies greatly depending on the transportation time and characteristics of the specific situation. We show that introducing the option of transportation enables makespan reductions of up to 10.91% which could greatly improve a

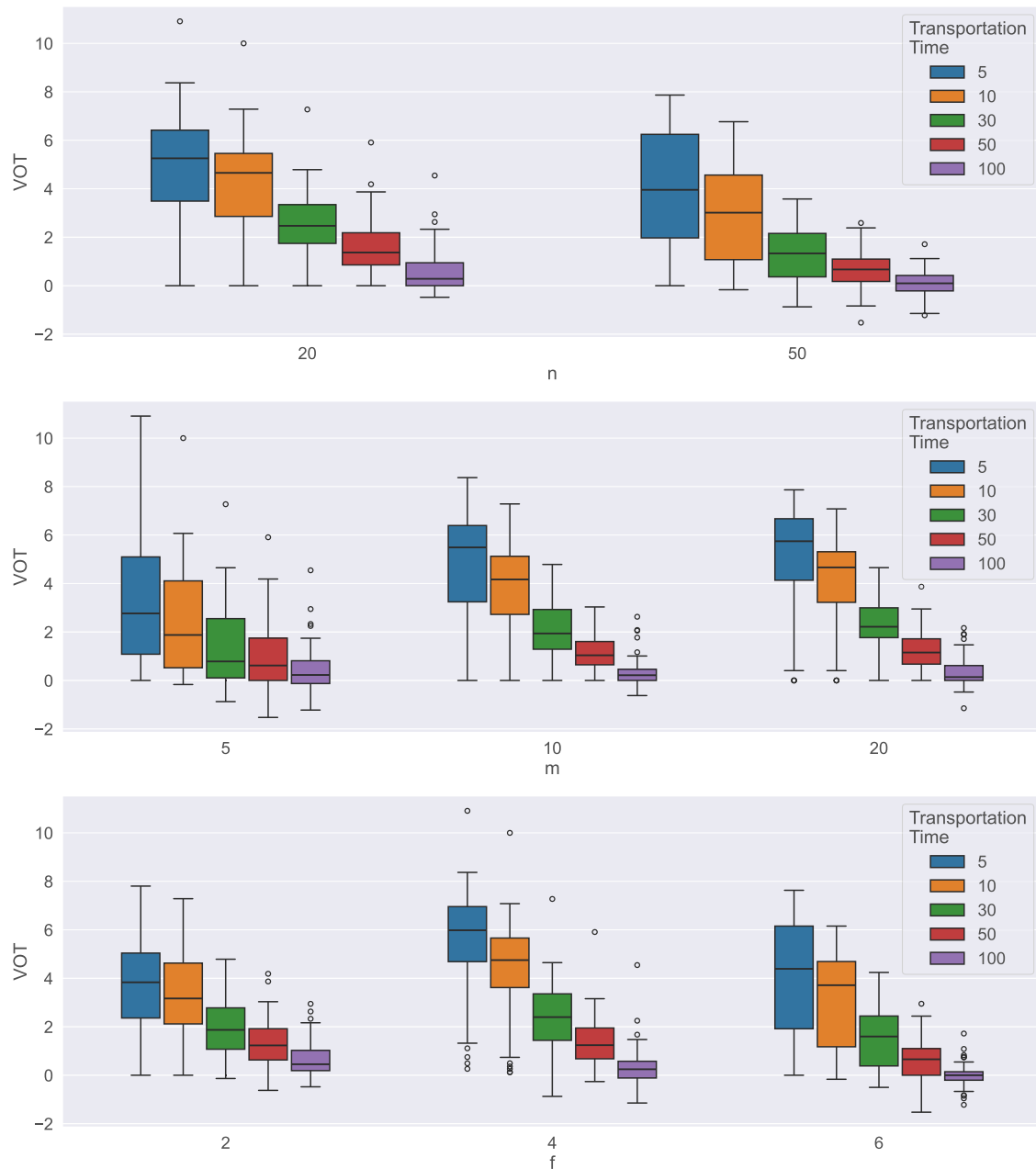


Fig. 8. Variation of the value of transportation (VOT: ARPD of DFSPT solution obtained by IG-TP vs. DPFSP solution) by number of jobs, machines, factories, and transportation time.

company's operational efficiency. Therefore, companies should evaluate the impact of transportation if they operate in a distributed flow shop environment.

However, we recognize several challenges connected to the transportation of intermediate goods between factories. One such challenge is the often substantial quantity of intermediate goods that must be moved between factories to realize the VOT. In many cases where the flow shops are located in close proximity or distributed across the same property, as is the case, e.g., for production modules in semiconductor industries, we believe such a schedule could be a valuable improvement. While companies typically have existing transportation infrastructures for receiving materials and shipping orders, the transportation infrastructure might have to be extended to allow for the increased demand of transporting intermediate goods. Therefore, each

transport also leads to economic and environmental costs in addition to the time requirement. When factories are dispersed across a larger geographical area and transport times are high, the complexity or cost of a schedule with inter-factory transportation might outweigh the benefits.

The DFSP is based on the premise that jobs can be freely assigned to one of the distributed flow shops. This suggests a predominant application to situations where factories are located in proximity and in turn, inter-factory transportation is a viable option. Furthermore, transportation introduces additional uncertainties to the distributed flow shop environment. These include, the possibility of damages during transit or delays, e.g., due to traffic, vehicle breakdowns, or weather conditions. Therefore, we recommend to consider the possibility of transportation carefully on a case-by-case basis.

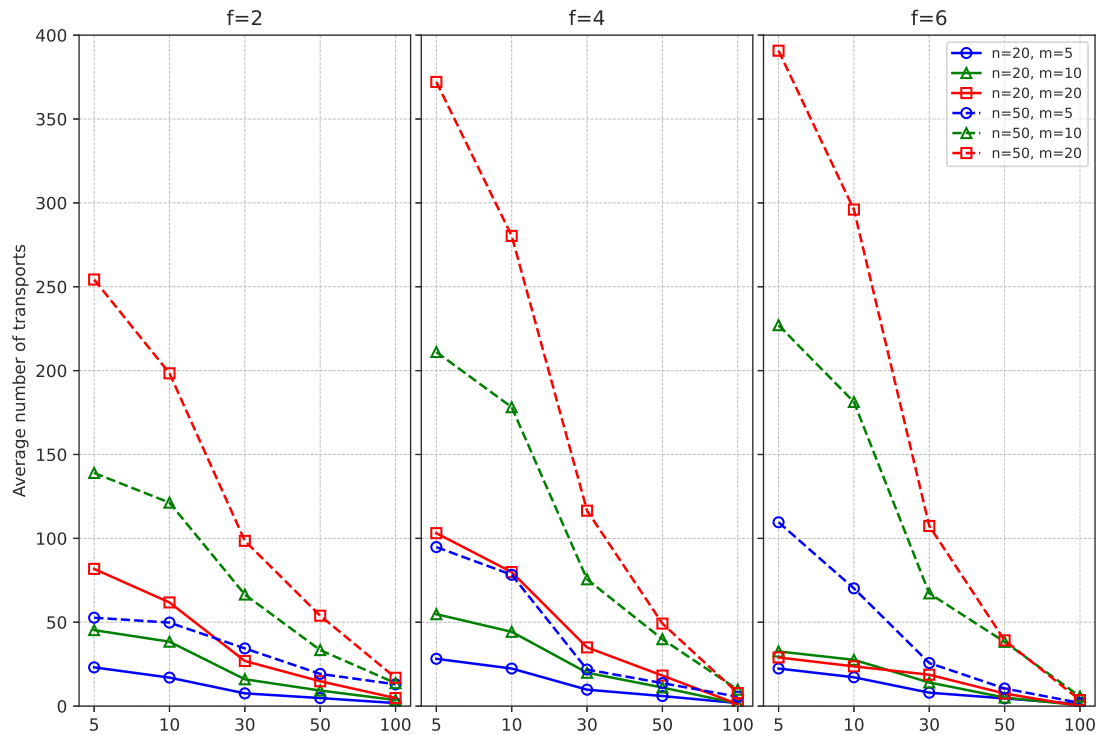


Fig. 9. Comparison of the average number of transports included in schedules across instance categories. A transport is defined as an operation changing factories between machines.

5. Conclusion and outlook

In this paper, we have introduced the Distributed Flow Shop Scheduling Problem with Transportation, a novel and more realistic problem setting that accounts for the transportation of jobs between machines. To address this complex problem, we have proposed the first heuristic algorithm based on the ideas of iterated greedy search. Additionally, we have presented a mathematical programming and constraint programming formulation. Our computational experiments have demonstrated the effectiveness of our heuristic algorithm and illustrate the potential of reducing makespan and in turn improving utilization when transportation between machines is possible as part of the distributed flow shop scheduling problem.

As the first work addressing the DFSPT, our study lays the foundation for further research in this area. Several avenues for future research can be explored. While our proposed algorithm exhibits a good performance, it would be interesting to explore the performance of other heuristic approaches, metaheuristics, or even exact methods to address the challenges posed by transportation in distributed flow shop environments. We recognize several additional objective functions that could be relevant for the DFSPT, e.g., the workload balance between factories, minimizing the number of transports, or tardiness-related objectives, which are highly relevant, for example, in semiconductor industries. Transportation of intermediate goods between factories could also introduce significant economical and environmental costs, as well as additional quality issues. Multi-objective approaches could provide a way to integrate these economic or environmental costs as additional objective functions. For other objectives, the impact of inter-factory transportation might be even higher compared to makespan minimization. Finally, several uncertainties arise with the possibility of transportation. These should be integrated in scheduling models for distributed manufacturing.

Table 5

Results for the ANOVA of the hyperparameter experiment for the operation-based destruction operator (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$).

Source	Sum of squares	Df	F-ratio	p-value
T	2.262515	2	4.1716	0.0158*
d	3.884901	2	7.1630	0.0008***
Residual	218.298671	805		

CRediT authorship contribution statement

Tristan Becker: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Janis Neufeld:** Conceptualization, Data curation, Formal analysis, Methodology, Writing – original draft, Writing – review & editing. **Udo Buscher:** Conceptualization, Data curation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing.

Acknowledgments

The authors gratefully acknowledge the computing time made available to them on the high-performance computer at the NHR Center of TU Dresden. This center is jointly supported by the Federal Ministry of Education and Research and the state governments participating in the NHR (www.nhr-verein.de/unsere-partner). Further, we would like to thank the anonymous referees whose thoughtful and extensive comments have greatly improved the quality of this work.

Appendix

See Tables 5–7.

Table 6
Results for the ANOVA of the hyperparameter experiment for the job-based destruction operator (* p < 0.05, ** p < 0.01, *** p < 0.001).

Source	Sum of squares	Df	F-ratio	p-value
T	41.875764	2	17.7609	0.0000***
d	8.535781	2	3.6203	0.0272**
Residual	948.995542	805		

Table 7
Results for the ANOVA of the hyperparameter experiment comparing the operation-based and job-based destruction operator (* p < 0.05, ** p < 0.01, *** p < 0.001).

Source	Sum of squares	Df	F-ratio	p-value
Destruction operator	116.8386	1	154.4670	0.0000***
Residual	1223.8532	1618		

References

Bargaoui, H., Driss, O. B., & Ghédira, K. (2017). A novel chemical reaction optimization for the distributed permutation flowshop scheduling problem with makespan criterion. *Computers & Industrial Engineering*, 111, 239–250.

Cai, S., Yang, K., & Liu, K. (2017). Multi-objective optimization of the distributed permutation flow shop scheduling problem with transportation and eligibility constraints. *Journal of the Operations Research Society of China*, 6(3), 391–416.

Chandra, R., Dagum, L., Kohr, D., Menon, R., Maydan, D., & McDonald, J. (2001). *Parallel programming in OpenMP*. Morgan Kaufmann.

Chen, J.-F., Wang, L., & Peng, Z.-P. (2019). A collaborative optimization algorithm for energy-efficient multi-objective distributed no-idle flow-shop scheduling. *Swarm and Evolutionary Computation*, 50, Article 100557.

Fernandez-Viagas, V. (2022). A speed-up procedure for the hybrid flow shop scheduling problem. *Expert Systems with Applications*, 187, Article 115903.

Fernandez-Viagas, V., & Framinan, J. M. (2014). A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 53(4), 1111–1123.

Fernandez-Viagas, V., Perez-Gonzalez, P., & Framinan, J. M. (2018). The distributed permutation flow shop to minimise the total flowtime. *Computers & Industrial Engineering*, 118, 464–477.

Fu, Y., Hou, Y., Chen, Z., Pu, X., Gao, K., & Sadollah, A. (2022). Modelling and scheduling integration of distributed production and distribution problems via black widow optimization. *Swarm and Evolutionary Computation*, 68, Article 101015.

Gao, J., & Chen, R. (2011a). A hybrid genetic algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Computational Intelligence Systems*, 4(4), 497–508.

Gao, J., & Chen, R. (2011b). An NEH-based heuristic algorithm for distributed permutation flowshop scheduling problems. *Scientific Research and Essays*, 6(4), 3094–3100.

Gao, J., Chen, R., & Deng, W. (2013). An efficient tabu search algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 51(3), 641–651.

Gao, J., Chen, R., Deng, W., & Liu, Y. (2012). Solving multi-factory flowshop problems with a novel variable neighbourhood descent algorithm. *Journal of Computational Information Systems*, 8, 2025–2032.

Garey, M. R., Johnson, D. S., & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2), 117–129.

Guo, H.-W., Sang, H.-Y., Zhang, B., Meng, L.-L., & Liu, L.-L. (2022). An effective meta-heuristic with a differential flight strategy for the distributed permutation flowshop scheduling problem with sequence-dependent setup times. *Knowledge-Based Systems*, 242, Article 108328.

Hamzadayi, A. (2020). An effective benders decomposition algorithm for solving the distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 123, Article 105006.

Han, B., Pan, Q.-K., & Gao, L. (2023). A cooperative iterated greedy algorithm for the serial distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 1–28.

Hou, Y., Fu, Y., Gao, K., Zhang, H., & Sadollah, A. (2022). Modelling and optimization of integrated distributed flow shop scheduling and distribution problems with time windows. *Expert Systems with Applications*, 187, Article 115827.

Li, H., Li, X., & Gao, L. (2021). A discrete artificial bee colony algorithm for the distributed heterogeneous no-wait flowshop scheduling problem. *Applied Soft Computing*, 100, Article 106946.

Li, Q., Li, J., Zhang, X., & Zhang, B. (2021). A wale optimization algorithm for distributed flow shop with batch delivery. *Soft Computing*, 25(21), 13181–13194.

Li, Y.-Z., Pan, Q.-K., Li, J.-Q., Gao, L., & Tasgetiren, M. F. (2021). An Adaptive Iterated Greedy algorithm for distributed mixed no-idle permutation flowshop scheduling problems. *Swarm and Evolutionary Computation*, 63, Article 100874.

Lin, S.-W., Ying, K.-C., & Huang, C.-Y. (2013). Minimising makespan in distributed permutation flowshops using a modified iterated greedy algorithm. *International Journal of Production Research*, 51(16), 5029–5038.

Lohmer, J., & Lasch, R. (2021). Production planning and scheduling in multi-factory production networks: a systematic literature review. *International Journal of Production Research*, 59(7), 2028–2054.

Mao, J.-Y., Pan, Q.-K., Miao, Z.-H., Gao, L., & Chen, S. (2022). A hash map-based memetic algorithm for the distributed permutation flowshop scheduling problem with preventive maintenance to minimize total flowtime. *Knowledge-Based Systems*, 242, Article 108413.

Midway, S., Robertson, M., Flinn, S., & Kaller, M. (2020). Comparing multiple comparisons: practical guidance for choosing the best multiple comparisons test. *PeerJ*, 8, Article e10387.

Moon, C., Kim, J., & Hur, S. (2002). Integrated process planning and scheduling with minimizing total tardiness in multi-plants supply chain. *Computers & Industrial Engineering*, 43(1–2), 331–349.

Naderi, B., & Ruiz, R. (2010). The distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 37(4), 754–768.

Naderi, B., Ruiz, R., & Roshanaei, V. (2023). Mixed-integer programming vs. constraint programming for shop scheduling problems: New results and outlook. *INFORMS Journal on Computing*, 35(4), 817–843.

Neufeld, J. S., Schulz, S., & Buscher, U. (2023). A systematic review of multi-objective hybrid flow shop scheduling. *European Journal of Operational Research*, 309(1), 1–23.

Pan, Q.-K., Gao, L., Wang, L., Liang, J., & Li, X.-Y. (2019). Effective heuristics and metaheuristics to minimize total flowtime for the distributed permutation flowshop problem. *Expert Systems with Applications*, 124, 309–324.

Perez-Gonzalez, P., & Framinan, J. M. (2023). A review and classification on distributed permutation flowshop scheduling problems. *European Journal of Operational Research*, 312(1–21), 278–285.

Ren, J., Ye, C., & Li, Y. (2021). A new solution to distributed permutation flow shop scheduling problem based on NASH Q-Learning. *Advances in Production Engineering & Management*, 16(3), 269–284.

Ribas, I., Companys, R., & Tort-Martorell, X. (2017). Efficient heuristics for the parallel blocking flow shop scheduling problem. *Expert Systems with Applications*, 74, 41–54.

Rifai, A. P., Nguyen, H.-T., & Dawal, S. Z. M. (2016). Multi-objective adaptive large neighborhood search for distributed reentrant permutation flow shop scheduling. *Applied Soft Computing*, 40, 42–57.

Ruiz, R., Pan, Q.-K., & Naderi, B. (2019). Iterated Greedy methods for the distributed permutation flowshop scheduling problem. *Omega*, 83, 213–222.

Ruiz, R., & Stützle, T. (2007). A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3), 2033–2049.

Schulz, S., Schönheit, M., & Neufeld, J. S. (2022). Multi-objective carbon-efficient scheduling in distributed permutation flow shops under consideration of transportation efforts. *Journal of Cleaner Production*, 365, Article 132551.

Shao, W., Pi, D., & Shao, Z. (2017). Optimization of makespan for the distributed no-wait flow shop scheduling problem with iterated greedy algorithms. *Knowledge-Based Systems*, 137, 163–181.

Taillard, E. (1990). Some efficient heuristic methods for the flow shop sequencing problem. *European Journal of Operational Research*, 47(1), 65–74.

Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2), 278–285.

Thoney, K. A., Hodgson, T. J., King, R. E., Taner, M. R., & Wilson, A. D. (2002). Satisfying due-dates in large multi-factory supply chains. *IEEE Transactions*, 34, 803–811.

Wang, H., Fu, Y., Huang, M., Huang, G. Q., & Wang, J. (2017). A NSGA-II based memetic algorithm for multiobjective parallel flowshop scheduling problem. *Computers & Industrial Engineering*, 113, 185–194.

Wang, K., Huang, Y., & Qin, H. (2016). A fuzzy logic-based hybrid estimation of distribution algorithm for distributed permutation flowshop scheduling problems under machine breakdown. *Journal of the Operational Research Society*, 67(1), 68–82.